Ella Weinberg and Michael Cinnamon (DAST Duo)

Info 415

Lindah Kotut

7 November, 2024

DAST Duo Comparison

Research questions

Why are DAST tools so important for the creation of web applications?

What are the main differences between the features of ZAP and Nikto?

What are the pros and cons of each DAST tool?

Introduction

Dynamic Application Security Testing tools are used by cybersecurity professionals and software developers to test the security of web applications. These tools allow organizations to identify and address vulnerabilities within their site before hackers can find and use them. With dozens of tools available, we wanted to gain a better understanding of the functionality and features that two popular and free tools provide.

DAST tools are becoming increasingly important in the world of cybersecurity as cyber-attacks continue to rise. One thing that these cyber-attacks target are vulnerabilities in web applications. Over 60% of applications are vulnerable to Cross-site scripting, 25% to broken authentication, 33% to sensitive data exposure, and 65% to SQL injection to name a few.[1] Successful XSS and SQL attacks as a result of web vulnerabilities can lead to information theft and financial loss. These DAST tools help teams identify these web weaknesses before attackers do.

We chose to use Zed Attack Proxy (ZAP) and Nikto for this demo because they are both free, open source, and claim to be leaders in web scanning technology. We also chose to use

[1] https://thelinuxcode.com/scanning_vulnerabilities_nikto/

ZAP due to its popularity; it claims to be the “the world’s most popular free open source scanner” that is used by professionals worldwide[2]. These two tools are well recommended by security professionals, and we thought it would be interesting to do a side-by-side comparison of scan results and interface that these tools offer. In our recorded tool demo we chose to analyze the scan results of a locally hosted password manager website that we own because DAST tests should be conducted on sites that have given permission to be attacked.

The Story

In the last few decades, the growth of web applications and scams that take place remotely has grown. This has led to a need for vulnerability detection tools which led to the development of DAST tools. ZAP was created and released in 2010 in an attempt to create an active community-based security project that made open-source security more accessible[3]. Since 2010 it has remained an open source project and still has a mission of growing cybersecurity collaboration efforts. It is now a GitHub top 1000 project that is downloaded and started over 1 million times per year[4]. Nikto has been in development since 1998 and released in 2001 and, like ZAP, is open source. It was written using the coding language Perl and can also be run as a docker container. It uses a large database of vulnerabilities that it checks during a scan and it continues to grow bigger every day.[5],[6]

---

[2] https://www.zaproxy.org/blog/2020-04-02-is-zap-the-worlds-most-popular-web-scanner/

[3] https://www.zaproxy.org/blog/2020-09-06-zap-is-ten-years-old/
[4] https://www.zaproxy.org/blog/2020-04-02-is-zap-the-worlds-most-popular-web-scanner/
[5]https://www.cybersecuriosity.com/blog/nikto-scanning-tool-a-comprehensive-guide-to-web-server-scanning
[6] https://github.com/sullo/nikto

ZAP Overview

ZAP Demo: https://www.youtube.com/watch?v=vUwBl3JzNc8

ZAP is a manipulator-in-the-middle proxy that intercepts packets sent between the tester browser and the web application. It can be downloaded for free on most systems and just requires Java 11 or higher to run on Windows and Linux systems[7]. After downloading and starting the ZAP app users can paste the URL of their target website, select the type of attack they want to perform, and ZAP will begin an automated scan. In this automated scan ZAP uses a spider (information gathering process) to scan all of the pages within the URL and uses an active scanner to attack all the discovered pages[8]. After running a scan ZAP provides a list of vulnerabilities and classifies them based on a severity rating of high, medium, low, or informational. Each alert found contains detailed information that maps the issue to a CWE, describes the vulnerability, and provides a recommendation. This output allows for users to prioritize and understand identified vulnerabilities.

Strengths	**Limitations**
Free and open-source	Scans can be slow depending on size of site
Most actively maintained project by OWASP[9]	Active scanning can cause damage to site functionality
App download/setup time takes minutes	Scanning spider can be ineffective on AJAX web applications
Large vulnerability detection library	User guide is dense and not beginner-friendly
Can perform crawling and active scans	Some professionals do not consider it a holistic security tool
Graphic dashboard interface that can be customized	Scans can slow down or crash computers

[7] https://www.zaproxy.org/getting-started/
[8] https://www.zaproxy.org/getting-started/
[9] https://brightsec.com/blog/dast-tools/

Customizable scan and reporting settings	
Available on Windows, Linux, and macOS	
Has a traditional spider and an AJAX spider	
Passive scanning is safe for the site and is done in the background or a site	
Comprehensive user guide and large community for support	
Features like fuzzing and proxying	

Nikto Overview

Nikto Demo: https://youtu.be/YImx8lSRzBk

Nikto is a free open-source web server scanner that scans the vulnerabilities of a given web application/website. It can be downloaded for free off of its website[10] or its GitHub[11] and only requires a terminal to run. To use it the user has to change the terminal directory to where Nikto is downloaded and type a simple line of code, which is given in a ReadMe file.[12] That line of code includes the file for the actual code, -h to specify a scan, and the website/application you want to scan.[13] Scan modifiers can also be added at the end of the code, such as -followredirects to go deeper into the website, -ssl to change from scanning HTTP to HTTPS, and -format to save the results to a specified file like CSV, HTML, and TXT.[14] The scan lists vulnerabilities as it finds them, which means you don't have to wait till it's done to start fixing the web server. One setting that can be tricky to find and can also hurt the scan is the error limit, if the scan comes across too many errors from the host it will stop the scan completely.[15] This tool

[10] https://cirt.net/Nikto2
[11] https://github.com/sullo/nikto
[12] https://github.com/sullo/nikto
[13] https://github.com/sullo/nikto
[14] https://github.com/sullo/nikto
[15] https://github.com/sullo/nikto/issues/549

is also not meant to be stealthy and will alert intrusion detection and prevention systems along with showing up in logs.[16]

Strengths	Limitations
Free and Open-source	Scans can take a while
Very easy to install as it comes in a zip folder	Terminal can be hard to read and hard to use for some
It's compatible with most(if not all) operating systems	Scans might be blocked by IPS/IDS
Can change config and other files to run your own personalized scans	Scans are not stealthy and
Only needs a terminal to run	Has an error limit that can end scans early
Gives a basic guide on how to use Nikto, including the scan modifiers	
Can save the output to a file or send it to another tool like Burp Suite and	
In-depth scans can tell a lot	
Large database of vulnerabilities	

Implications

DAST tools can be used by malicious actors, red teams, and blue teams to better understand a web application and possible exploitable areas. Blue teams can use ZAP to help ensure compliance with OWASP recommendations and can use it for configuration checks of servers and databases, code reviews, and to simulate attacks to understand how they may happen.

ZAP in particular is helpful for testing applications for compliance with security standards like OWASP Top 10 and the severity levels and detailed guidance help organizations prioritize remediation efforts effectively. This alignment with OWASP Top 10 is important because OWASP sets a sort of industry standard for web app security and cybersecurity professionals

[16] https://cirt.net/Nikto2

can use this as a baseline for best security practices. ZAP's alignment with the OWASP Top 10 can help security teams be compliant for regulatory audits such as the Payment Card Industry Data Security Standard[17]. These DAST tools allow for faster detection of vulnerabilities and can be used early in the creation of a dynamic website which can help reduce attack surfaces. These tools are effective for early detection of flaws and can save companies money, as many tools are free, and fixing vulnerabilities reduces the risk of hacker exploitation and damage to a system[18].

Output Comparison

The output below shows our findings from each of the three scans (on a local host site, My Plan, and a bank demo site) we conducted as well as a breakdown of which vulnerabilities were unique to these tools. Nikto seemed to pick up on issues related to server setup and legacy configurations, while ZAP focused more on application-layer vulnerabilities and missing defensive measures.

---

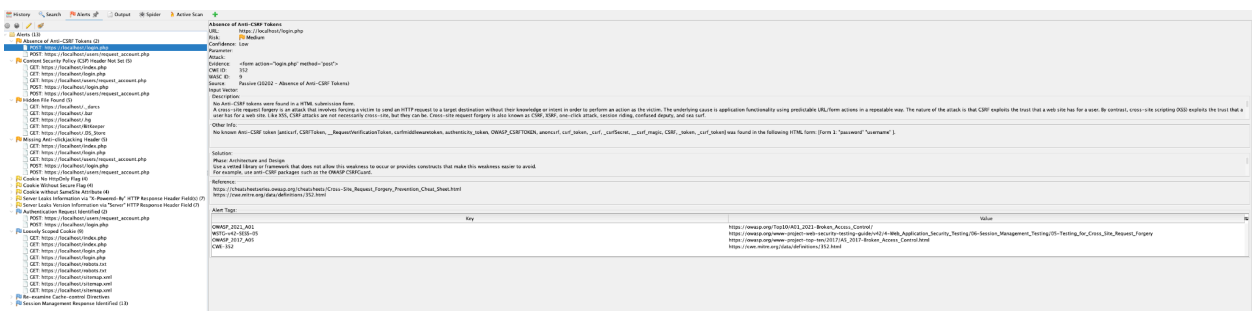
[17] https://www.securityjourney.com/post/beyond-the-owasp-top-10-for-companies-subject-to-pci-compliance

[18] https://blueteamalpha.com/blog/web-application-scanning-a-beginners-guide/

The two screenshots below provide a side-by-side comparison of the output that ZAP and Nikto produced after scanning our locally hosted password manager website.

```
Last login: Fri Nov 22 15:22:19 on ttys002
ellaweinberg@Ellas-MacBook-Air-18 ~ % cd Downloads/nikto-master/program
ellaweinberg@Ellas-MacBook-Air-18 program % ./nikto.pl -h https://localhost/login.php -followredirects
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          127.0.0.1
+ Target Hostname:    localhost
+ Target Port:        443
---------------------------------------------------------------------------
+ SSL Info:           Subject:  /C=US/ST=WA/L=SEA/O=UW/CN=localhost/OU=iSchool
                      Altnames: localhost
                      Ciphers:  AEAD-CHACHA20-POLY1305-SHA256
                      Issuer:   /C=US/ST=Washington/O=UW/OU=iSchool
+ Start Time:         2024-11-22 15:25:23 (GMT-8)
---------------------------------------------------------------------------
+ Server: nginx/1.27.0
+ /login.php: Retrieved x-powered-by header: PHP/7.4.33.
+ /login.php: Uncommon header(s) 'x-content_type-options' found, with contents: nosniff.
+ /login.php: Cookie PHPSESSID created without the secure flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /login.php: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /login.php: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ /login.php: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ /login.php: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /login.php/sips/sipssys/users/a/admin/user: SIPS v0.2.2 allows user account info (including password) to be retrieved remotely. See: https://vulners.com/exploitdb/EDB-ID:22381
+ /login.php/_vti_bin/shtml.dll/_vti_rpc?method=server+version%3a4%2e0%2e2%2e2611: Gives info about server settings. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-0413,http://cve.mitre.org/c
gi-bin/cvename.cgi?name=CVE-2000-0709,http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-0710
+ /login.php/_vti_bin/shtml.exe/_vti_rpc?method=server+version%3a4%2e0%2e2%2e2611: Gives info about server settings. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-0413,http://cve.mitre.org/c
gi-bin/cvename.cgi?name=CVE-2000-0709,http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-0710
+ /login.php/_vti_bin/_vti_aut/author.dll?method=list+documents%3a3%2e0%2e2%2e1706&service%5fname=&listHiddenDocs=true&listExplorerDocs=true&listRecurse=false&listFiles=true&listFolders=true&listLinkInfo=
true&listIncludeParent=true&listDerivedT=false&listBorders=false: We seem to have authoring access to the FrontPage web.
+ /login.php/_vti_bin/_vti_aut/author.exe?method=list+documents%3a3%2e0%2e2%2e1706&service%5fname=&listHiddenDocs=true&listExplorerDocs=true&listRecurse=false&listFiles=true&listFolders=true&listLinkInfo=
true&listIncludeParent=true&listDerivedT=false&listBorders=false: We seem to have authoring access to the FrontPage web.
+ /login.php/protected/secret.html+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /login.php/.htpasswd: Contains authorization information.
+ /login.php/radio.php: PHP backdoor found.
+ 12482 requests: 0 error(s) and 16 item(s) reported on remote host
+ End Time:           2024-11-22 15:54:40 (GMT-8) (1757 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
ellaweinberg@Ellas-MacBook-Air-18 program %
```

Screenshot of output from Nikto scan of locally hosted website.



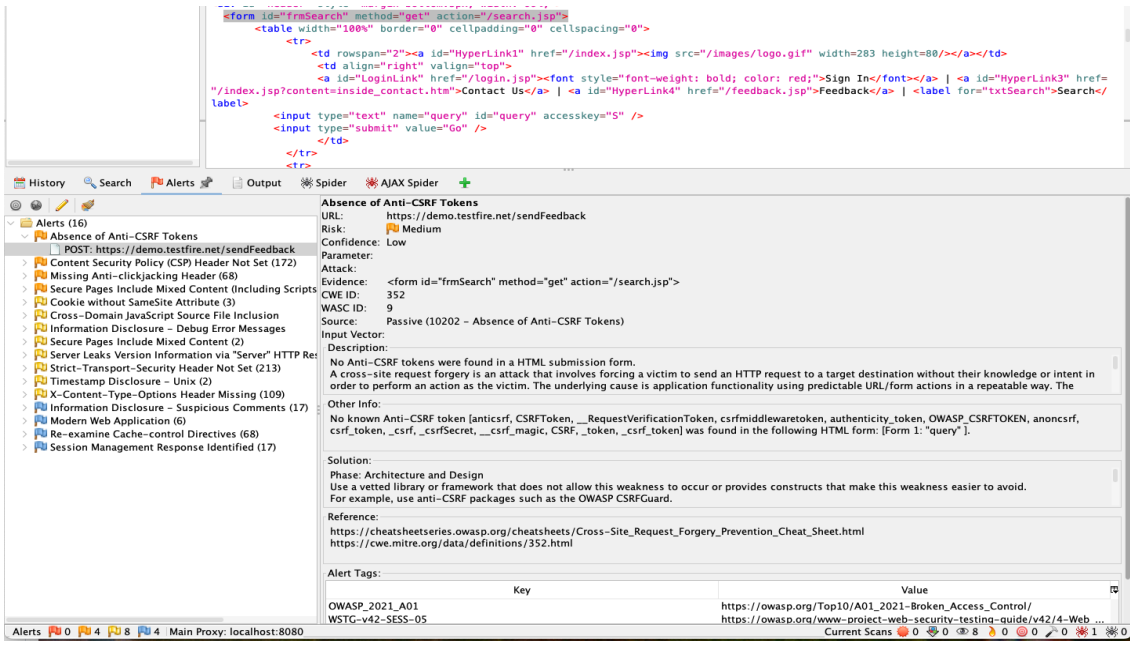
Screenshot of output from ZAP of locally hosted website.

Comparison of tool output:

Zap	Both	Nikto
No anti-CSRF token	Security headers not set	No CGI directories
Anti-clickjacking header	Cookies without certain flags (HttpOnly, Secure)	PHP Backdoor
Cookie without SameSite attribute	Server HTTP problems	Uncommon headers found
Loosely scoped cookie	Some hidden files	Authoring Access to FrontPage

Authentication Request identified		Account info can be accessed remotely
Re-examine cache control directives		

The two screenshots below provide a side-by-side comparison of the output that ZAP and Nikto produced after scanning the bank demo website.



Screenshot of the output from running Zap on the banking website

```
Kingm@DESKTOP-GQ6G7DU MINGW64 ~/downloads/nikto/program
$ ./nikto.pl -h https://demo.testfire.net -followredirects
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          65.61.137.117
+ Target Hostname:    demo.testfire.net
+ Target Port:        443
---------------------------------------------------------------------------
+ SSL Info:           Subject:  /CN=demo.testfire.net
                      Altnames: demo.testfire.net, altoromutual.com
                      Ciphers:  ECDHE-RSA-AES256-GCM-SHA384
                      Issuer:   /C=GB/ST=Greater Manchester/L=Salford/O=Sectigo Limited/CN=Sectigo RSA Domain Validation Secure Server CA
+ Start Time:         2024-11-07 16:07:23 (GMT-8)
---------------------------------------------------------------------------
+ Server: Apache-Coyote/1.1
+ /: The site uses TLS and the Strict-Transport-Security HTTP header is not defined. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-sca
nner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, PUT, DELETE, OPTIONS .
+ HTTP method ('Allow' Header): 'PUT' method could allow clients to save files on the web server.
+ HTTP method ('Allow' Header): 'DELETE' may allow clients to remove files on the web server.
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /admin/config.php: PHP Config file may contain database IDs and passwords.
+ /admin/cplogfile.log: DevBB 1.0 final log file is readable remotely. Upgrade to the latest version. See: http://www.mybboard.com
+ /admin/system_footer.php: myphpnuke version 1.8.8_final_7 reveals detailed system information.
+ /admin/admin_phpinfo.php4: Mon Album version 0.6.2d allows remote admin access. This should be protected.
+ /admin/login.php?action=insert&username=test&password=test: phpAuction may allow user admin accounts to be inserted without proper authentication. Attempt to log in with user 'test' password 'test' to verify.
See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2002-0995
+ /admin/contextAdmin/contextAdmin.html: Tomcat may be configured to let attackers read arbitrary files. Restrict access to /admin. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-0672
+ //admin/admin.shtml: Axis network camera may allow admin bypass by using double-slashes before URLs. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-0240
+ /admin/database/wwForum.mdb: Web Wiz Forums pre 7.5 is vulnerable to Cross-Site Scripting attacks. Default login/pass is Administrator/letmein. See: OSVDB-2813
+ //admin/aindex.htm: FlexWATCH firmware 2.2 is vulnerable to authentication bypass by prepending an extra /'s. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2006-3604
+ /admin/wg_user-info.ml: WebGate Web Eye exposes user names and passwords. See: OSVDB-2922
+ /admin/: This might be interesting.
+ /bank/: This might be interesting.
+ /admin/auth.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/cfg/configscreen.inc.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/cfg/configsite.inc.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/cfg/configsql.inc.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/cfg/configtache.inc.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/cms/htmltags.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/credit_card_info.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/exec.php3: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/index.php: This might be interesting: has been seen in web logs from an unknown scanner.
```

```
+ /admin/cfg/configsite.inc.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/cfg/configsql.inc.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/cfg/configtache.inc.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/cms/htmltags.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/credit_card_info.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/exec.php3: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/index.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/modules/cache.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/objects.inc.php4: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/script.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/settings.inc.php+: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/templates/header.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/upload.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ /admin/adminproc.asp: Xpede administration page may be available. The /admin directory should be protected. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2002-0579
+ /admin/datasource.asp: Xpede page reveals SQL account name. The /admin directory should be protected. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2002-0579
+ /admin/admin.php?adminpy=1: PY-Membres 4.2 may allow administrator access. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2001-1198
+ /admin/account.asp: Admin login page/section found.
+ /admin/account.html: Admin login page/section found.
+ /admin/account.php: Admin login page/section found.
+ /admin/controlpanel.asp: Admin login page/section found.
+ /admin/controlpanel.html: Admin login page/section found.
+ /admin/controlpanel.php: Admin login page/section found.
+ /admin/cp.asp: Admin login page/section found.
+ /admin/cp.html: Admin login page/section found.
+ /admin/cp.php: Admin login page/section found.
+ /admin/home.asp: Admin login page/section found.
+ /admin/home.php: Admin login page/section found.
+ /admin/index.asp: Admin login page/section found.
+ /admin/index.html: Admin login page/section found.
+ /admin/login.asp: Admin login page/section found.
+ /admin/login.html: Admin login page/section found.
+ /admin/login.php: Admin login page/section found.
+ /admin/html: Tomcat Manager / Host Manager interface found (pass protected).
+ /admin/status: Tomcat Server Status interface found (pass protected).
```

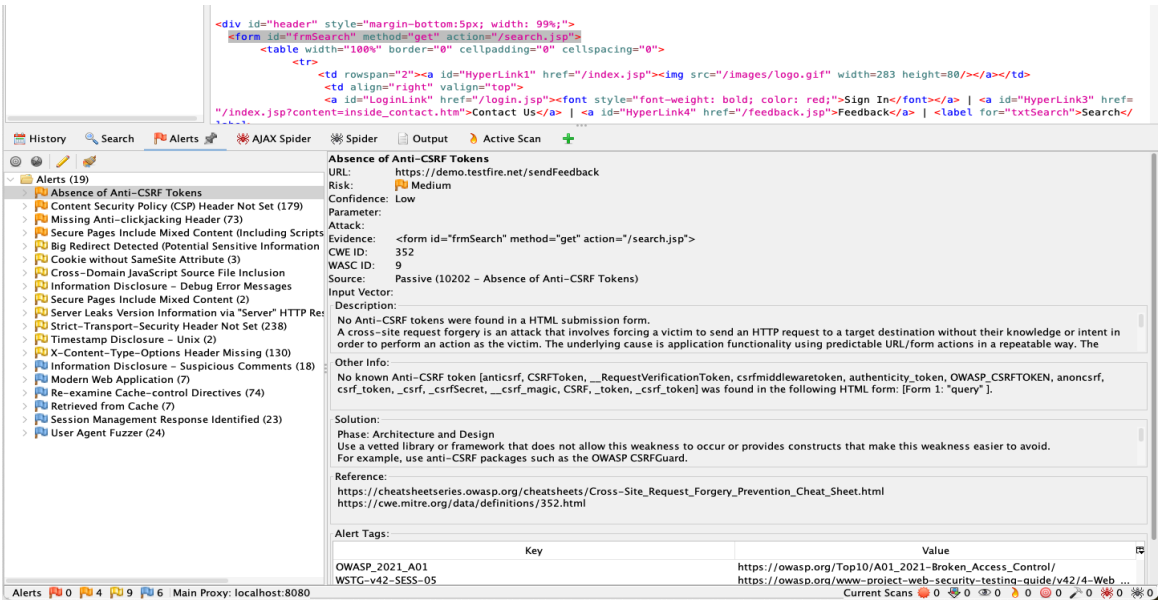
Screenshot of the output from running Nikto on the banking website

Comparison of tool output:

ZAP	Both	Nikto
Absence of Anti-CSRF Tokens	Security headers missing	The site uses TLS
Content Security Policy (CSP) Header Not Set	Strict-Transport-Security Header Not Set	HTTP methods allowed (Get, head, post, put, delete, options)

Missing Anti-clickjacking Header	X - Content - Type - Options Header Missing	Web server returns a valid response with junk HTTP
Secure Pages Include Mixed Content		PHP Config may contain IDs and passwords
Cookie without SameSite Attribute		Version Specific issues * DevBB Final log file is readable remotely
Cross-Domain JavaScript Source File Inclusion		Myphpnuke reveals detailed server information
Information Disclosure - Debug Error Messages		Mon album allows remote admin access
Secure Pages Include Mixed Content		Web Wiz Forums are vulnerable to Cross-site scripting
Server Leaks Version Information vis “Server” HTTP Requests		Flexwatch vulnerable to authentication bypass
Timestamp Disclosure		WebGate Web Eye exposes usernames and passwords
Information Disclosure - Suspicious Comments		Axis network camera allows admin bypass by using double //
Modern Web Application		Tomcat configured to read arbitrary files
Re-examine Cache-control Directives		phpAuction allows user admin accounts to be inserted without proper authentication
		Xpede reveals SQL account name
		Py-Membres may allow admin access *
		Marks around 25 ‘interesting’ things

The two screenshots below provide a side-by-side comparison of the output that ZAP and Nikto produced after scanning the MyPlan UW page.



Screenshot of output from ZAP on MyPlan

```
Kingm@DESKTOP-GQ6G7DU MINGW64 ~/downloads/nikto/program
$ ./nikto.pl -h https://myplan.uw.edu -followredirects
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Multiple IPs found: 18.172.170.19, 18.172.170.87, 18.172.170.97, 18.172.170.116
+ Target IP:          18.172.170.19
+ Target Hostname:    myplan.uw.edu
+ Target Port:        443
---------------------------------------------------------------------------
+ SSL Info:           Subject:  /CN=myplan.uw.edu
                      Altnames: myplan.uw.edu, myplan.washington.edu
                      Ciphers:  TLS_AES_128_GCM_SHA256
                      Issuer:   /C=US/O=Amazon/CN=Amazon RSA 2048 M02
+ Start Time:         2024-11-07 15:41:47 (GMT-8)
---------------------------------------------------------------------------
+ Server: AmazonS3
+ /home: Retrieved via header: 1.1 06a6a5c4cebb71ab27b2d7f062023104.cloudfront.net (CloudFront).
+ /home: Uncommon header(s) 'x-amz-error-message' found, with contents: Resource Found.
+ /home: Uncommon header(s) 'x-amz-error-code' found, with contents: Found.
+ /home: The site uses TLS and the Strict-Transport-Security HTTP header is not defined. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ Root page / redirects to: /home/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ : Server banner changed from 'AmazonS3' to 'CloudFront'.
+ /home: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ /home: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ /home: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ /home: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ /home: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ /console/: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerabi
lity-scanner/vulnerabilities/missing-content-type-header/
+ /console: This might be interesting.
+ /home/: This might be interesting.
+ /soap/servlet/soaprouter: Oracle 9iAS SOAP components allow anonymous users to deploy applications by default. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2001-1371
+ /console/: Application console found.
+ 8344 requests: 215 error(s) and 15 item(s) reported on remote host
+ End Time:           2024-11-07 15:49:50 (GMT-8) (483 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Screenshot of output from Nikto on MyPlan

Comparison of tool output:

Zap	Both	Nikto
Anti-CSRF Token	Security Headers not set	Uncommon Headers

Missing anti-clickjacking header	Strict-Transport-Security HTTP header not set	Root page redirects
Secure pages include Mixed content	X-Content-Type-Options header not set	Oracle allows users to deploy applications by default
Big Redirect detected		Application console found
Cookie without SameSite attribute		Server banner changed
Debug error messages		No CGI directories
Cross-domain JavaScript		Couple of interesting pages
Server leaks server information		
Timestamp disclosure		
User-agent fuzzer		

Takeaways

While both tools are trusted and reliable for identifying vulnerabilities in web applications, ZAP is more beginner-friendly as it has a graphic user interface rather than producing command-line output like Nikto. While the ZAP output is easier to read and clearly maps issues to pages within the web application, it has a lot of opportunities for plugins and customizable settings that may make it initially overwhelming for users to understand.

With that being said, different scanners test for slightly different vulnerabilities, so it can be beneficial for security teams/developers to use more than one scanner. As shown above, while there was some overlap in results for our scans, ZAP scanned more for OWASP vulnerabilities and Nikto scanned more for server configurations. ZAP found application-layer vulnerabilities that are found by interrupting the flow of packets sent from the browser and server. Nikto found more server-side vulnerabilities that can be useful in understanding the overall attack surface of a site.

These two tools provide slightly different layers of coverage and using both of them (or a collection of multiple DAST tools is the best approach that security professionals can take to strengthen an organization's web security.